SPOT-The-Difference: Latent-Aware Causal Structure Learning for Microservice Latency on PetShop

Milestone 2 — Technical Foundation

Anant Narayan, Ankit Kumar, Milan Shah

CSE516 – Probabilistic Graphical Models, Ahmedabad University

ABSTRACT

Milestone 1 proposed evaluating Skeleton Posterior-guided Optimization (SPOT) against its ABIC backbone on the Sachs dataset. For Milestone 2 we fixed one mode (Applied AI) and one domain: causal structure learning for microservice latency on the PetShop benchmark, where the true call graph, once inverted, gives a ground-truth causal graph and where many services are unobserved, so latent confounding arises naturally. This report documents the technical foundation: a PetShop data pipeline with a latent-aware ground truth (inverted call graph → latent projection → MAG → PAG), a dependency-free re-implementation of ABIC and SPOT on one shared differentiable optimizer with analytic gradients, a skeleton-posterior model trained on simulated graphs (held-out AUROC 0.989), the paper's skeleton/arrowhead/tail F1 metrics with a runtime breakdown, and, new in this version, a root-cause inference layer with a written justification of the ADMG. On three of the four PetShop scenarios (partial: 2–3 optimizer seeds, one restart) SPOT with a calibrated pruning floor raises mean skeleton F1 from 0.514 (ABIC) to 0.550, better in 3 of 3 scenarios but within optimizer-seed spread, at 0.42× ABIC's optimization time; arrowhead and tail differences are inside that spread, and both learners return far more bidirected edges (17–41) than the ground truth (0–1). The posterior, trained on simulations, is out of distribution on PetShop (AUROC 0.83–0.93). On simulated raw data SPOT clearly beats ABIC (skeleton F1 0.961 vs 0.894) but only marginally on z-scored data. A root-cause inference layer is implemented and unit-tested on PetShop-format data; its evaluation on the real issue runs is planned for M3.

Index Terms*—causal discovery, latent confounders, ADMG, MAG, PAG, skeleton posterior, SPOT, ABIC, PetShop, microservices*

Table I
M2 DELIVERABLES AND STATUS

Deliverable	Content	Status
Dataset pipeline	PetShop loader, node rule, preprocessing, latent-aware ground truth	Done
Graphical model	Linear-Gaussian ancestral ADMG; PAG for scoring	Done
Learning pipeline	ABIC, SPOT, skeleton posterior, shared optimizer	Done
Baseline	ABIC (same optimizer, no mask); FCI/RFCI	ABIC done; FCI not run
Inference pipeline	Root-cause ranking from the learned ADMG; ADMG justification	Implemented and unit-tested; real-run evaluation in M3
Initial KPIs	F1/TPR/FDR, runtime	Synthetic done; PetShop partial (3 of 4 scenarios, 2–3 seeds)

I. INTRODUCTION AND CHANGES SINCE M1

A. Scope of this milestone

The course guidelines ask an Applied-AI M2 for a dataset pipeline, a graphical model, a learning pipeline, an inference pipeline, a baseline, and initial KPI results. Our status against each item is summarised in Table I; we state plainly which items are complete, partial or not started.

B. What changed from M1

M1 proposed the Sachs protein-signaling data with variable-removal experiments. Two problems led us to change the domain. First, the Sachs reference graph is a biological consensus, not a ground truth, and its dimensionality (11 variables) leaves little room for the scalability argument that motivates SPOT. Second, we wanted a domain in which latent variables are a fact of the data rather than something we delete by hand. PetShop [5] fits: it records latency, request and availability metrics for a deployed AWS microservice application together with its call graph, and some components have no usable metric stream. The rest of the M1 plan is unchanged: SPOT versus ABIC at equal budget, the paper's F1 metrics, reproducibility over seeds, and an honest treatment of what is identifiable. Sachs is dropped from the scope; the M1 hypotheses H1–H4 are re-targeted at PetShop and at the simulated graphs.

C. Problem statement

Given latency time series from the services of a microservice application, in which some services are unobserved, learn a graph that exposes which services' latency drives which, and which pairs share an unobserved cause. Operators use such a graph for root-cause analysis of latency incidents: a slow database should be traced to the callers it slows down, and two services slowed together by an unmeasured shared dependency should not be mistaken for one causing the other. The learned graph must therefore (i) distinguish direct causal edges from confounding and (ii) support probabilistic queries, which is what the inference layer in Section V addresses.

II. DATASET AND DATA PIPELINE

A. PetShop

PetShop [5] is an openly licensed benchmark (data CC-BY-4.0, code Apache-2.0) generated from an adapted AWS observability demo application. It provides four scenarios (`low_traffic`, `high_traffic`, `temporal_traffic1`, `temporal_traffic2`), each with a call graph (`graph.csv`, caller→callee), metrics for normal operation (`noissue`), and injected-issue runs with root-cause labels. Metrics are latency, request count and availability, with several statistics each. We use the *latency, average* series of the no-issue runs for structure learning, because latency is the quantity whose propagation

Table II
PETSHOP SCENARIOS AFTER OUR PIPELINE

Scenario	n	Obs. d	Latent	$\rightarrow$	$\leftrightarrow$	PAG edges
low_traffic	589	29	13	30	1	31
high_traffic	589	35	9	38	0	38
temporal_traffic1	1652	31	10	32	1	33
temporal_traffic2	1652	31	10	32	1	33

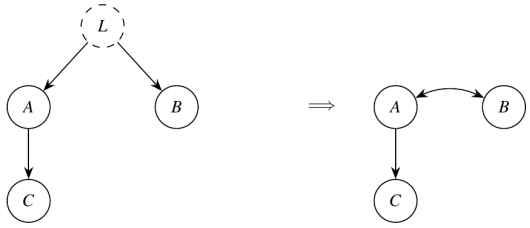


Fig. 1. Latent projection. If the callee L (dashed) has no usable metrics, its two callers A and B become a bidirected pair in the ADMG; the observed edge $A \rightarrow C$ is kept.

along calls is the causal story; the injected-issue runs are used only by the root-cause inference layer (Section V).

B. Ground truth from the inverted call graph

Latency of a callee propagates to its callers, so the causal graph is the call graph with every edge reversed (callee→caller), as prescribed for this project. Not every node of the call graph yields a usable metric series. We call a node *observed* if it has a latency column, at most 5% missing values and non-constant values; all other nodes are treated as *latent*. We then build the ground truth in four steps:

1. reverse the call graph to obtain a DAG $\mathcal{G}$ over all nodes $V \cup L$;
2. apply Verma–Pearl latent projection onto the observed nodes V, giving an ADMG: $a \rightarrow b$ if a directed path with all interior nodes latent leads from a to b, and $a \leftrightarrow b$ if some latent node has such paths to both;
3. take the maximal ancestral graph (MAG) of the ADMG by the inducing-path rule [6];
4. convert the MAG to its PAG with Tetrad's MAG-to-PAG routine (the `causal-cmd` jar shipped with the SPOT artifact) [7].

Scoring against the PAG, not the MAG, follows the SPOT paper: arrowheads and tails that no method can determine from observational data are left as circles and are not counted as errors.

Table II lists the resulting problem sizes. Two features matter for interpretation. The ground truth is almost entirely directed edges; with only 0–1 bidirected edges, arrowhead and tail scores largely measure directed-edge recovery. And $d \approx 30$ with n between 589 and 1652 is the regime where ABIC is already slow (Section VII-B).

C. Preprocessing and its caveats

Remaining gaps are linearly interpolated. The default transform is Gaussian-rank (quantile) followed by centring and unit-variance scaling; log and raw variants are available. Latency is right-skewed, so the rank transform brings the margins closer to the Gaussian assumption of the model, at the cost of distorting scale relationships. Samples are treated as i.i.d. even though the data are time series; all scenarios show lag-1 autocorrelation of about 0.3 (mean over services, after rank transformation), and we expect conditional-independence tests to be over-confident there. This is a violated assumption, recorded here.

III. GRAPHICAL MODEL

A. What is learned

Both methods learn a *linear-Gaussian structural equation model with correlated errors*, parameterised by a directed weight matrix D and a symmetric error-covariance matrix Ω whose off-diagonal pattern is the bidirected part:

$$\mathbf{X} = D^\top \mathbf{X} + \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, \Omega), \quad \Sigma = (I - D^\top)^{-1} \Omega (I - D^\top)^{-\top}. \tag{1}$$

The support of D gives directed edges and the off-diagonal support of Ω gives bidirected edges, which together form an acyclic directed mixed graph (ADMG) restricted to the *ancestral* class: a bidirected edge may not join a variable to one of its ancestors. Ancestrality guarantees that the ADMG has a corresponding MAG, which is what permits scoring against a PAG. A bidirected edge stands for one or more unmeasured common causes, never for a cycle or a measured relation.

B. Why an ADMG suits this domain

A DAG over the observed services would have to explain the shared slowdown of A and B in Fig. 1 by a direct edge between them, which is false and would misdirect a root-cause search. The ADMG keeps what is measured as directed edges and records the unmeasured part as $A \leftrightarrow B$. The graph also matches how operators reason: latency flows against the call direction, a service has a few downstream dependencies, and shared dependencies such as a database or load balancer are often unmonitored. The ADMG is the exact marginal of a DAG with hidden nodes, so it is the smallest graph class that stays correct when some nodes are unobserved; and the benchmark's own truth exists as an ADMG/MAG only because 9–13 nodes per scenario are latent.

Our results also weaken the practical argument, and we say so. After projection the truth contains only 0–1 bidirected edges per scenario, so PetShop is a *weak test* of the benefit of confounding-aware modelling; and our learners return many more bidirected edges than that (Section VII-D), so bidirected edges in learned graphs are mostly artefacts. Whether using the bidirected part improves root-cause ranking is therefore an empirical question that our inference layer is built to answer but has not yet answered on real data. Section VIII lists where the linear-Gaussian form is a poor fit.

C. Representation, inference, learning (project mapping)

Representation: ADMG over observed services with parameters (D, Ω); skeleton posterior $P = (p_{ij})$ as an intermediate object; PAG for scoring only. *Inference*: Gaussian marginal and conditional queries from Σ, and mechanism-shift root-cause scores (Section V). *Learning*: constrained maximum likelihood by the optimizer of Section IV, with SPOT adding the posterior. The skeleton posterior is not a Bayesian network over services and contains no latent nodes; latent confounding appears only through Ω.

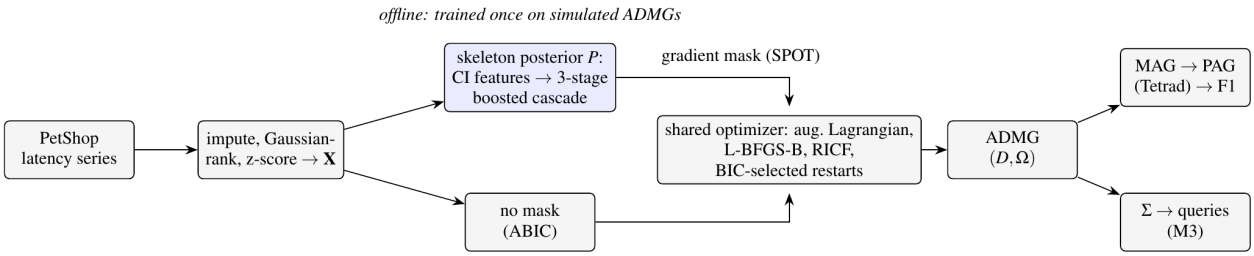


Fig. 2. Pipeline. ABIC and SPOT call the same optimizer; the only difference is the gradient mask derived from the skeleton posterior P. The box "$\Sigma \to$ queries" is the inference layer of Section V, now implemented (root-cause ranking); evaluation on the issue runs follows in M3.

IV. Learning Pipeline

A. ABIC: the baseline and the backbone

ABIC [3] minimises a least-squares data term built from the RICF pseudo-variables, plus a smooth ℓ_0 penalty, subject to $h(D,\Omega) = 0$ where h combines an acyclicity term on D and an ancestrality term that penalises bidirected weight between ancestor pairs. The constraint is enforced by an augmented Lagrangian ($\frac{\rho}{2}h^2 + \alpha h$); each subproblem is solved by L-BFGS-B with box bounds, RICF is interleaved to refresh the error-covariance pseudo-variables, and the best of several random restarts is chosen by BIC. After optimisation, small weights are thresholded, the threshold being raised until the directed support is acyclic. Our implementation follows the released code's penalty (NOTEARS-polynomial cycle term and truncated-series ancestrality term) and its smooth-ℓ_0 scale; the matrix-exponential form of the paper is available as an option. Gradients are analytic and were checked against central finite differences (maximum relative error about 10^{-9}, both penalty forms, including pinned coordinates). RICF was checked against direct maximum likelihood (agreement within 10^{-3} nats) and recovers generating parameters at $n = 20{,}000$. A Gram-matrix form of the data term cut the cost of a value-and-gradient call at $d = 30$ from 2.1 ms to 0.63 ms without changing its value (difference 4×10^{-15}).

B. Skeleton posterior

SPOT's first stage estimates $p_{ij} = q_\phi(S_{ij} = 1 \mid X)$ with an amortised model trained offline on simulated data [4]. Our simulator draws random ancestral ADMGs with 25–40 nodes, linear-Gaussian parameters, n log-uniform in $[300, 2000]$, and optional AR(1) noise ($\rho \in \{0, 0.3, 0.6\}$) because PetShop is a time series; each instance is labelled with the skeleton of the true MAG. Features are summaries (four moments and eleven percentiles) of Fisher-z conditional-independence statistics over conditioning sets of order 0, 1 and 2, plus Markov-blanket features. Three gradient-boosting classifiers are stacked in a cascade, each receiving out-of-fold predictions of the previous one. The paper uses XGBoost; XGBoost is not installable in our environment, so we use scikit-learn's `HistGradientBoostingClassifier`, the same model family but not identical numbers.

C. SPOT: posterior-guided optimisation

At every function evaluation the gradient g of the augmented-Lagrangian objective is masked per coordinate. For a coordinate θ_k of vertex pair (i, j): if $\theta_k g_k > 0$ (the step moves θ_k towards zero, a "weakening" move) the gradient is kept; otherwise ("strengthening") it is kept with probability $(p_{ij} + c)^{1/(t+1)}$, $c = 0.1$, t the outer iteration, and zeroed if rejected. As t grows the probability tends to 1 and SPOT approaches ABIC. The released code additionally pins pairs with $p_{ij} \le 10^{-3}$ to zero through the box bounds; we keep this as a switch (released floor) and ablate it. Because our boosted cascade is less extreme than the released XGBoost one, 10^{-3} prunes little; we therefore also report a calibrated floor of 6×10^{-3}, chosen as the largest value that pins at most 1% of true edges on simulated graphs disjoint from every benchmark graph (0.93% on held-out seed 777), never tuned on benchmark data.

D. Fairness protocol

ABIC and SPOT call the same function with the same loss, penalty, L-BFGS settings, restarts, thresholds and seeds; SPOT differs only by the mask (and optional floor). Setting the posterior to $p \equiv 1$ without floor reproduces ABIC bit for bit, which is a unit test. When a wall-clock budget is used, SPOT's online posterior time is deducted from it. Runtime is reported as three numbers: offline training (one time), online posterior inference, and optimisation.

E. Differences from the released code

We re-implemented rather than ran the released artifact, because it depends on packages we cannot install here (XGBoost, causal-learn, ananke, autograd). During this work we found two issues in the released code that we report as findings, not as invalidation of the paper.

Inverted p-value test. In `learn/gen_feature.py`, the order-0 screen keeps a pair when the Fisher-z p-value is greater than α and the Markov-blanket step adds a variable when the test statistic exceeds α; a small p-value means dependence, so read literally this selects independent pairs. We only read the source. Table III shows its effect in our re-implementation on 40 held-out simulated graphs: the screen keeps 98.1% of true edges when corrected but 1.9% when inverted, yet end-to-end AUROC differs only by about 0.012 (paired SE 0.0017, better on 36 of 40 graphs) because the later cascade stages re-derive adjacency. We use the corrected direction; the inverted one remains available as a diagnostic.

Metric indexing. The released `compare_pag` indexes arrowheads and tails by the pair (node2, node2), so only the endpoint, not the edge, is compared. We report corrected metrics (identified by the ordered node pair) as primary and the bug-compatible values as "legacy" alongside.

Table III
EFFECT OF THE p-VALUE DIRECTION (40 HELD-OUT GRAPHS)

Variant	Screen keeps	AUROC	AUPRC
corrected ($p \leq \alpha$)	98.1%	0.988	0.936
legacy ($p > \alpha$), same design	1.9%	0.976	0.915
legacy, released design	1.9%	0.975	0.903

V. INFERENCE PIPELINE

Given fitted (D, Ω), Σ in Eq. (1) defines a joint Gaussian over the observed services, so marginal and conditional latency queries (for example the distribution of a front-end service's latency given downstream latencies) are closed-form Gaussian conditioning. The component we implemented and tested is *root-cause ranking* (module `pgm/rca.py`), which uses the PetShop issue runs.

Setting. A metric of the target service (PetSite) violates its SLO. We have normal data (`noissue`) and abnormal data (rows of the issue file at or after the injection timestamp) for all services, and must rank services by how likely each is the injected root cause. Only latency issues are scored.

Method. Data are mapped by a transform fitted on the normal data only (log by default; z-score and Gaussian-rank available). The ADMG is fitted to the normal data by RICF. A root cause is a service whose own mechanism changed, modelled as a mean shift of its structural noise. With $\Delta\mu$ the abnormal mean shift of the observed vector, the noise-space shift is $\Delta\varepsilon = (I - D^\top)\Delta\mu$, which is near zero for a service that merely transmits an upstream anomaly. The score of service i is the reduction in Mahalanobis distance obtained by allowing a shift at i alone (generalised least squares):

$$s_i = \frac{(\Omega^{-1}\Delta\varepsilon)_i^2}{(\Omega^{-1})_{ii}}. \tag{2}$$

Three variants share data, transform and candidate set: *anomaly* (no graph, $s_i = \Delta\mu_i^2$), *dag* (directed edges only, Ω diagonal) and *admg* (directed plus bidirected, full Ω). Running *dag* and *admg* on the same graph isolates what the bidirected part buys, because with bidirected edges Ω^{-1} discounts the part of a shift explained by noise shared with a confounded neighbour. The target itself is never a candidate; an optional filter restricts candidates to directed ancestors of the target. Graph sources are ABIC, calibrated SPOT, and the ground-truth MAG as a reference upper bound that is unavailable in practice. Labels are never used to learn graphs or set parameters. The planned metrics are top-1 and top-3 recall and mean reciprocal rank, per scenario and pooled, with the published PetShop numbers (graph given, different protocol) as context only.

Status. The pipeline, the issue-file loader and the summarizer are written. The stage-3 notes report five passing unit tests on synthetic PetShop-format data (identity of the score with the GLS form, recovery of a shifted mechanism on a true ADMG, helper functions, loader and node-name matching, end-to-end runner); that data is deliberately easy, so the tests check logic and plumbing, not accuracy. Testing found and fixed a timestamp-conversion bug that silently selected zero abnormal rows. A first pass over the real `low_traffic` issue files found 14 latency issues (4 train, 10 test); scoring them is part of M3. Assumptions not yet checked against the real files are: field layout (verified against the PetShop paper, not the files), matching of node names in `target.json` to graph nodes, and the abnormal window (the SLO violation may lag the injection). In the temporal scenarios, load varies over the day, so normal and abnormal windows can differ by load alone.

VI. BASELINES AND EVALUATION PROTOCOL

A. Baselines

ABIC, as the unguided backbone, is the primary baseline and is implemented. FCI/RFCI via the Tetrad bridge are a constraint-based comparison that we have not yet run; we do not report any FCI numbers here. Ablations are implemented: SPOT without the floor (mask only), floor without the mask, a random posterior with the same probabilities assigned to random pairs, an oracle posterior built from the true skeleton, and the second released variant (sign-blind scaling). They have been run on synthetic graphs only, not on PetShop.

B. Metrics

Following the SPOT paper, we report F1, TPR and FDR for the skeleton, arrowheads and tails of the estimated ADMG's PAG against the true PAG. The estimated ADMG is converted by the same MAG-then-PAG route as the ground truth. Skeleton, arrowhead and tail are separate columns because F1 for orientation marks is meaningful only where the PAG commits to a mark.

VII. RESULTS

A. Component validation

The skeleton posterior reaches held-out AUROC 0.989, AUPRC 0.946, log-loss 0.057 and ECE 0.013 on 60 freshly simulated graphs; 95.7% of true edges get $p \geq 0.05$. Offline training took 203 s on one core (CI features 152 s, classifier fitting 51 s); online inference takes 0.29–0.32 s per dataset. The paper reports AUROC 0.99 at 50–100 nodes; the gap in AUPRC and KL should not be over-read because graph sizes, settings and the KL definition differ.

B. Cost of the baseline

ABIC time is dominated by the number of L-BFGS solves rather than by the cost of one evaluation. On z-scored data ($n = 500$, one restart, converged to $h \leq 10^{-8}$), one restart takes about 4–17 s at $d = 10$, 67–71 s at $d = 20$ and 163–171 s at $d = 30$. A z-scored PetShop run with $d = 29$–35 therefore costs minutes per restart per method, and the benchmark uses one restart and a common protocol.

C. Synthetic graphs: SPOT versus ABIC

Table IV compares the methods on simulated ancestral ADMGs with $d = 25$, $n = 1000$, one restart and six graphs per setting, with the posterior of Section IV-B (trained on graphs that exclude these). Raw data are the paper's simulator output; z-scored data are the setting that PetShop preprocessing resembles.

Raw data. Calibrated-floor SPOT improves skeleton F1 by $+0.067$, winning on 6 of 6 graphs (Wilcoxon $p = 0.031$, the smallest value possible with six pairs) and arrowhead F1 by $+0.110$ (5 wins, 1 loss, $p = 0.062$), in $0.32\times$ ABIC's total time.

Table IV
SYNTHETIC, $d = 25$, $n = 1000$, 6 GRAPHS (MEAN ± SD)

Method	Skel. F1	Arrow F1	Tail F1	Time (s)
Raw data (paper's simulator)				
ABIC	.894±.091	.806	.704	75.7
SPOT, floor 10^{-3}	.949±.033	.878	.751	55.5
SPOT, floor 6×10^{-3}	.961±.017	.916	.892	24.0
z-scored data				
ABIC	.687±.087	.390	.104	82.9
SPOT, floor 10^{-3}	.718±.020	.387	.117	96.1
SPOT, floor 6×10^{-3}	.730±.077	.419	.106	59.4

The released-floor variant gains +0.055 (4 wins, 2 losses, $p = 0.16$). This reproduces the SPOT paper's qualitative claim in our implementation, with large per-graph timing variance (ABIC 33–130 s).

z-scored data. Calibrated-floor SPOT improves skeleton F1 by +0.042, winning on 5 of 6 graphs ($p = 0.094$); released-floor SPOT gains +0.031 (4 wins, 2 losses, $p = 0.56$). Neither is significant. Arrowhead and tail F1 show no reliable difference. Both methods return about 27 bidirected edges where the truth has 1–4: on z-scored data the least-squares pseudo-variable term counts a bidirected edge in both endpoints' equations, so descent prefers bidirected explanations unless the ancestrality penalty forbids them. The truth has lower BIC than what ABIC returns, so this is a bias of the objective plus local optima, not a gradient bug, and the released objective matches ours. Raw simulated data are also var-sortable (downstream variances grow), which a scale-dependent loss can exploit; PetShop is not, so we treat the z-scored rows as the better guide.

Runtime. Released-floor SPOT was 1.32× slower than ABIC on z-scored data and calibrated-floor SPOT 0.80×, consistent with the modest speed-up (3139 s to 2883 s, about 8%) in the SPOT paper's Table 7. Almost every L-BFGS solve hits the 1000-iteration cap, so the mask cannot shorten solves much; the saving comes from pruning, with 84 of 300 pairs left free under the calibrated floor. Under an equal 60 s budget on four z-scored graphs, ABIC reaches skeleton F1 0.668 (converged on 1 of 4) and calibrated SPOT 0.689 (converged on 2 of 4), better on 3 of 4 graphs ($p = 0.625$); not conclusive.

Fig. 3 shows the control experiments on three graphs. A random posterior (same probabilities on random pairs) drops skeleton F1 to 0.33, so the posterior's information matters; the oracle posterior reaches 0.92 skeleton F1 in about a quarter of ABIC's time, so the headroom is large. The gap between the oracle and SPOT comes from pairs the learned posterior leaves free, not from a failing mask. On those same three graphs, skeleton F1 is 0.635 for ABIC, 0.722 for released-floor SPOT, 0.688 for calibrated-floor SPOT, 0.671 for mask only, 0.667 for floor only and 0.624 for the second released variant. These differences are within graph-to-graph variation, so the ablations do not yet say how much of any gain is the stochastic mask and how much is pruning.

D. PetShop benchmark (partial)

We ran ABIC, released-floor SPOT and calibrated-floor SPOT on `low_traffic` and `high_traffic` (seeds 0–2) and

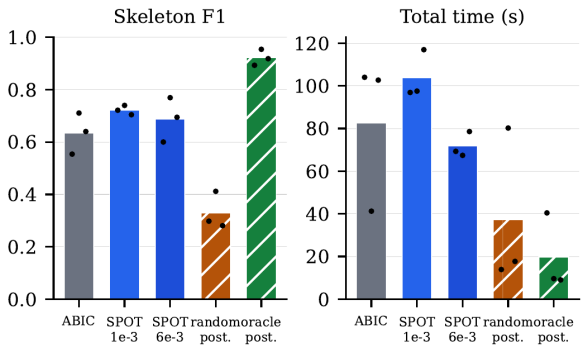


Fig. 3. Controls on three graphs (z-scored, $d = 25$). Bars are means; dots are individual graphs. Random posterior is far worse than ABIC; oracle posterior is best and fastest.

Table V
PETSHOP: F1 AVERAGED OVER 3 SCENARIOS

Method	Skeleton F1	Arrowhead F1	Tail F1
ABIC	0.514	0.301	0.056
SPOT, floor 6×10^{-3}	0.550	0.317	0.105
SPOT, floor 10^{-3}	0.535	0.297	0.063

`temporal_traffic1` (seeds 0–1), with Gaussian-rank and z-scored data, one restart, no wall-clock budget and the stored posterior model; all 24 runs converged and none hit a time limit. `temporal_traffic2`, seeds 3–4, the PetShop ablations, equal-budget runs and the raw-centred sensitivity run were *not run*. PetShop is one dataset per scenario, so seeds vary only the optimizer: ± is optimizer variance, not data variance, and with three scenarios we report mean differences and wins/losses/ties over scenarios and make no significance claim. These runs supersede the single `low_traffic` run of the previous draft of this report.

Accuracy. Differences to ABIC (wins/losses/ties over scenarios): calibrated SPOT +0.036 (3/0/0) skeleton, +0.016 (2/1/0) arrowhead, +0.049 (2/0/1) tail; released-floor SPOT +0.021 (3/0/0), −0.004 (2/1/0), +0.007 (1/1/1). The skeleton-F1 gain of SPOT over ABIC is positive in all three scenarios (+0.036 calibrated, +0.021 released) but of the same order as the optimizer-seed spread (Table VI), so it is tentative. Arrowhead and tail differences are inside that spread, and tail F1 is low for every method (scenario means 0.056–0.105; exactly zero on `low_traffic`). Corrected and legacy metrics differ greatly: legacy tail F1 is 0.27 for ABIC against 0.06 corrected, because the legacy convention compares only which node carries a mark; legacy values are in the result files.

Bidirected over-use. Both learners return 17–41 bidirected edges and only 6–14 directed edges per run against 0–1 bidirected and 30–38 directed in the truth, so arrowhead and tail scores mostly measure how directed edges are mislabelled as bidirected. This is the over-use seen on z-scored synthetic data (Section VII-C), with a larger ratio.

Runtime. Optimization time per run (run-weighted over all eight runs per method) is 115 s for ABIC, 46 s for calibrated SPOT (0.42×) and 58 s for released-floor SPOT (0.53×), with 97,916, 40,331 and 49,784 L-BFGS evaluations per run on average. To this SPOT adds a one-time offline posterior training

Table VI
PETSHOP PER SCENARIO (MEAN ± SD OVER OPTIMIZER SEEDS)

Scenario (n/d, seeds)	Method	Skeleton F1	Skel. TPR	Skel. FDR	Arrow F1	Tail F1	Est. $\rightarrow$ / $\leftrightarrow$ (true)	Opt. time (s)
low (589/29, 3)	ABIC	.484±.019	.49±.02	.52±.02	.288±.018	.000±.000	7.3 / 25.0 (30 / 1)	83
	SPOT, 6×10^{-3}	.521±.028	.51±.04	.46±.01	.243±.015	.000±.000	9.3 / 19.7	19
	SPOT, 10^{-3}	.514±.017	.51±.02	.47±.04	.298±.013	.000±.000	9.3 / 20.7	32
high (589/35, 3)	ABIC	.513±.018	.61±.04	.56±.01	.299±.012	.167±.059	12.3 / 39.3 (38 / 0)	192
	SPOT, 6×10^{-3}	.537±.014	.60±.03	.51±.00	.301±.059	.181±.166	14.3 / 32.0	78
	SPOT, 10^{-3}	.536±.019	.62±.03	.53±.02	.258±.014	.123±.003	13.3 / 37.0	92
temporal 1 (1652/31, 2)	ABIC	.544±.004	.61±.00	.51±.01	.317±.054	.000±.000	6.5 / 34.0 (32 / 1)	47
	SPOT, 6×10^{-3}	.590±.019	.62±.02	.44±.02	.407±.024	.133±.000	8.5 / 28.0	39
	SPOT, 10^{-3}	.554±.017	.58±.03	.47±.01	.334±.010	.067±.067	7.5 / 28.0	46

Table VII
SKELETON POSTERIOR APPLIED TO PETSHOP (STORED MODEL)

Data	AUROC	AUPRC	Pinned true edges
PetShop low_traffic	0.830	0.515	32%
PetShop high_traffic	0.931	0.590	16%
PetShop temporal_traffic1	0.876	0.570	15%
simulated held-out graphs	0.989	0.946	≈1%

Pinned = share of true edges set to zero by the 6×10^{-3} floor.

Table VIII
REMAINING WORK

Milestone	Work
M3 (1 Nov)	Run root-cause evaluation on the real PetShop issue runs (top-1/top-3 recall, MRR; anomaly vs. dag vs. admg; ABIC, SPOT and truth graphs); complete `temporal_traffic2`, seeds 3–4, PetShop ablations, equal-budget runs and the raw-vs-z-scored sensitivity run; FCI/RFCI baseline; statistical tests; error analysis of missed and spurious edges; posterior suited to PetShop-like data; video presentation.
M4 (22 Nov)	Final tables and plots, robustness, limitations, reproducibility package, justification that the ADMG is a good representation of microservice latency (written argument exists; empirical support pending), final SOTA positioning.

(203 s) and 0.32 s of online inference per dataset. The saving tracks the number of free pairs (105 under the calibrated floor, 198 under the released floor, all pairs for ABIC), which points to pruning rather than the stochastic mask, but no PetShop ablation separates the two (the synthetic ablation could not either), so we say "tracks", not "is caused by". The saving is small on `temporal_traffic1` (47 vs. 39–46 s). Timings are single-core and should be compared as ratios.

Why the posterior helps little. Table VII shows the posterior was trained on linear-Gaussian simulations and PetShop is out of distribution: its AUROC against the true skeleton is 0.83–0.93 rather than 0.989, and AUPRC about 0.52–0.59. On `low_traffic` the calibrated floor pins 32% of true edges to zero, against ≈1% on simulations, which caps SPOT's recall. (An earlier posterior retrained in this environment gave AUROC 0.84–0.92 on all four scenarios, so the conclusion does not depend on the particular model.) PetShop's sample sizes and mean lag-1 autocorrelation (0.31–0.35 after rank transformation) are inside the training ranges, so the gap is not a simple range mismatch; the skewed, non-Gaussian and call-graph-shaped structure is the likelier cause, which we have not yet isolated. A skeleton posterior trained on PetShop-like simulations, or the paper's dynamic simulator with domain adaptation, is the natural remedy.

VIII. LIMITATIONS AND RISKS

- **Out of distribution.** The posterior is trained on linear-Gaussian simulated graphs; PetShop is skewed, autocorrelated and call-graph shaped, and its AUROC there is 0.83–0.93 against 0.989 held-out on simulations (Table VII). The calibrated floor, tuned on simulations, is too aggressive on PetShop.
- **Violated assumptions.** Non-Gaussian margins and i.i.d. treatment of time series affect both methods; absolute F1 may be low for both, while the comparison between them stays valid. Users retrying slow pages create feedback that a DAG/ADMG cannot represent, and in the temporal scenarios the load changes over the day.
- **Bidirected over-use.** Both methods overuse bidirected edges (17–41 against 0–1 true), which depresses arrowhead and tail F1 and may hurt, as well as help, a root-cause score that uses Ω.
- **Small numbers.** Six synthetic graphs (three for ablations) and, on PetShop, one dataset per scenario with 2–3 optimizer seeds cannot establish significance; we report paired wins/losses and make no claims beyond them.
- **One restart.** Runs use one restart because of cost; BIC-selected restarts are known to matter.
- Inference layer: unit-tested on PetShop-format data; its accuracy on the real issue runs, and the contribution of the bidirected part, are M3 questions.
- **Ground-truth caveat.** The inverted call graph states what can influence what by design; it is not guaranteed to equal the statistical dependence structure of the measured series.

IX. PLAN FOR M3 AND M4, RESPONSIBILITIES

Responsibilities. *Anant Narayan*: SPOT implementation and this report. *Ankit Kumar*: benchmarks (PetShop and synthetic runs, ablations, result tables). *Milan Shah*: inference layer (root-cause ranking and the written ADMG justification) and the video presentation. All members integrate, cross-review and reproduce runs. Code was written with generative-AI assistance; the individual submissions state each member's own use.

X. Conclusion

We have a working technical foundation: a latent-aware PetShop ground truth, ABIC and SPOT on one verified optimizer, a skeleton posterior with strong held-out accuracy, the paper's metrics with a corrected indexing, and a root-cause inference layer. The evidence so far is mixed. SPOT beats ABIC clearly on raw simulated data, but on z-scored data resembling PetShop's preprocessing the skeleton gain is small and non-significant. On three PetShop scenarios SPOT is better than ABIC on skeleton F1 in every scenario ($+0.021$ to $+0.036$ on average) and about 2–$2.4\times$ faster per run, within optimizer-seed spread, with no reliable arrowhead or tail gain, and the posterior is clearly out of distribution there. The inference layer is in place; in M3 we run the root-cause evaluation, completing the remaining scenarios and seeds and PetShop ablations, and improve the posterior for PetShop-like data.

References

[1] J. Pearl, *Causality: Models, Reasoning, and Inference*, 2nd ed. Cambridge, U.K.: Cambridge Univ. Press, 2009.

[2] X. Zheng, B. Aragam, P. Ravikumar, and E. P. Xing, "DAGs with NO TEARS: Continuous optimization for structure learning," in *Adv. Neural Inf. Process. Syst.*, vol. 31, 2018, pp. 9492–9503.

[3] R. Bhattacharya, T. Nagarajan, D. Malinsky, and I. Shpitser, "Differentiable causal discovery under unmeasured confounding," in *Proc. 24th Int. Conf. Artif. Intell. Statist.*, vol. 130, 2021, pp. 2314–2322.

[4] P. Ma, R. Ding, Q. Fu, J. Zhang, S. Wang, S. Han, and D. Zhang, "Scalable differentiable causal discovery in the presence of latent confounders with skeleton posterior," in *Proc. 30th ACM SIGKDD Conf. Knowl. Discovery Data Mining*, 2024, pp. 2148–2159.

[5] M. Hardt, W. R. Orchard, P. Blöbaum, E. Kirschbaum, and S. Kasiviswanathan, "The PetShop dataset — Finding causes of performance issues across microservices," in *Proc. 3rd Conf. Causal Learning and Reasoning*, ser. PMLR, vol. 236, 2024, pp. 957–978.

[6] T. Richardson and P. Spirtes, "Ancestral graph Markov models," *Ann. Statist.*, vol. 30, no. 4, pp. 962–1030, 2002.

[7] J. Zhang, "On the completeness of orientation rules for causal discovery in the presence of latent confounders and selection bias," *Artif. Intell.*, vol. 172, no. 16–17, pp. 1873–1896, 2008.